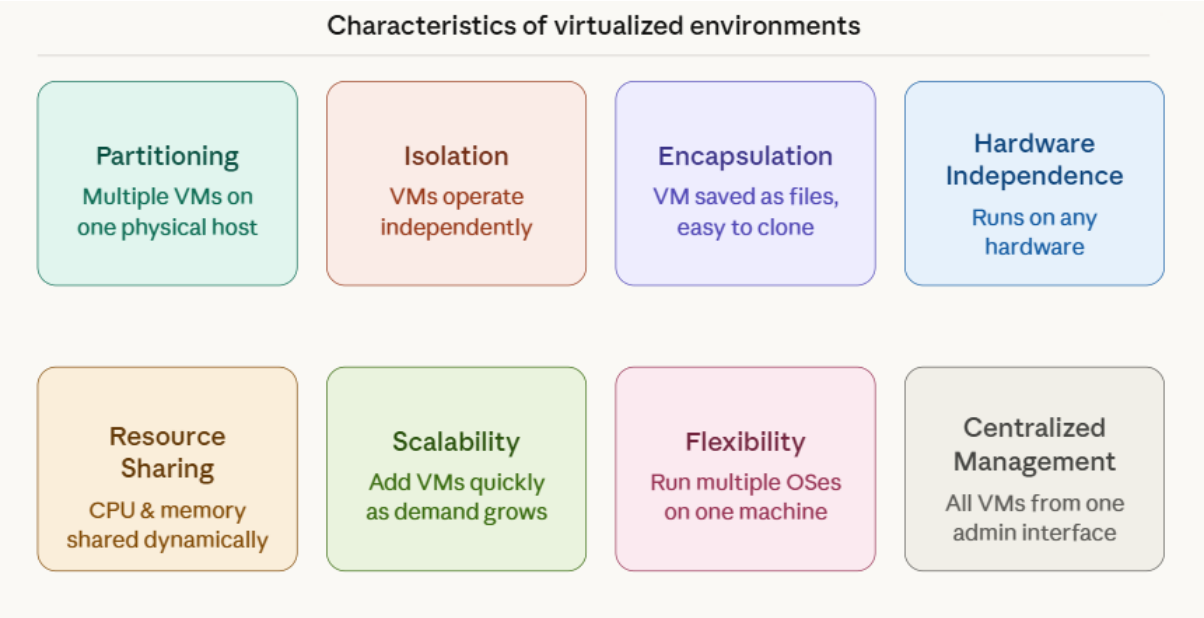
UNIT – 3: VIRTUALIZATION AND CONTAINERS

Virtualization:

Virtualization is the process of creating a virtual version of computing resources such as hardware, storage, or network using a virtualization layer (hypervisor).

CHARACTERISTICS OF VIRTUALIZED ENVIRONMENTS

Virtualization means creating a virtual version of computing resources (like servers, storage, or networks) using software.



1. Partitioning

- A single physical system is divided into multiple virtual machines (VMs).
- Each VM runs its own operating system independently.
- Resources like CPU, memory are allocated to each VM separately.
- Helps in running multiple applications on the same hardware efficiently.

2. Isolation

- Each VM operates independently from others.
- Failure or crash in one VM does not affect other VMs.
- Provides strong security between environments.
- Prevents unauthorized access between virtual machines.

3. Encapsulation

- A VM is stored as a set of files on disk.
- Easy to copy, move, or clone VMs.
- Simplifies backup and recovery processes.
- Enables quick restoration using snapshots.

4. Hardware Independence

- VMs are not dependent on specific physical hardware.

- Can run on different machines without modification.
- Supports migration across systems easily.
- Reduces compatibility issues with hardware.

5. Resource Sharing

- Physical resources are shared among multiple VMs.
- CPU, memory, and storage are allocated dynamically.
- Improves overall system utilization.
- Reduces hardware wastage and cost.

6. Scalability

- New VMs can be created quickly as needed.
- Resources can be increased or decreased dynamically.
- Supports growing workloads efficiently.
- Useful in cloud and large-scale systems.

7. Flexibility

- Multiple operating systems can run on a single machine.
- Supports different applications simultaneously.
- Useful for testing, development, and deployment.
- Easy to modify configurations based on needs.

8. Centralized Management

- All VMs can be managed from a single interface.
- Simplifies monitoring and control of systems.
- Reduces administrative complexity.
- Enables automation of tasks like deployment and updates.

TAXONOMY OF VIRTUALIZATION TECHNIQUES

Taxonomy means classification of virtualization methods based on how virtualization is implemented.

1.Execution Environment

1.1. Full Virtualization

- Entire hardware is simulated using a hypervisor.
- Guest OS runs without any modification.
- Provides complete isolation between virtual machines.
- Slight performance overhead due to emulation.

Advantages:

- Runs unmodified guest OS (no changes required).
- Provides strong isolation between virtual machines.

Disadvantages:

- **Performance overhead** due to full hardware simulation.
- Requires **more system resources**.

1.2. Para-Virtualization

- Guest OS is modified to interact with hypervisor.
- Improves communication between OS and hardware.
- Better performance compared to full virtualization.

- Requires changes in operating system.

Advantages:
- **Better performance** due to direct interaction with hypervisor.
- **Efficient resource utilization**.

Disadvantages:
- Requires **modification of guest OS**.
- Limited support for all operating systems.

1.3. Hardware-Assisted Virtualization
- Uses processor features like Intel VT-x and AMD-V.
- No need to modify guest operating system.
- Improves efficiency and execution speed.
- Reduces virtualization overhead.

Advantages:
- **High performance** using CPU support (Intel VT-x, AMD-V).
- No need to modify the guest OS.

Disadvantages:
- Requires **special hardware support**.
- May increase system **complexity and cost**.

2. OS-Level Virtualization (Containerization)
- Multiple containers share the same OS kernel.
- Lightweight and fast startup.
- Efficient resource utilization.
- Limited to same operating system.

Advantages:
- **Lightweight and fast startup**.
- **Efficient resource usage** (no full OS needed).

Disadvantages:
- All containers must use the **same OS kernel**.
- Provides **less isolation** compared to VMs.

3. Application Virtualization
- Applications run in isolated virtual environments.
- No need to install directly on OS.
- Avoids conflicts between applications.
- Easy deployment and portability.

Advantages:
- No need for **installation on host OS**.
- Avoids **software conflicts**.

Disadvantages:
- Limited compatibility with some applications.
- May have **performance issues** for heavy apps.

4. Network Virtualization
- Combines hardware and software network resources.
- Creates multiple virtual networks on same infrastructure.

- Improves flexibility and scalability.
- Enhances network security and management.

Advantages:

- Improves **network flexibility and scalability**.
- Enhances **security through isolation**.

Disadvantages:

- Complex to implement and manage.
- Requires **advanced networking knowledge**.

5. Storage Virtualization

- Combines multiple storage devices into a single unit.
- Simplifies storage management.
- Improves storage utilization.
- Provides scalability and flexibility.

Advantages:

- Simplifies **storage management**.
- Improves **storage utilization**.

Disadvantages:

- Initial setup can be **complex**.
- Possible **performance bottlenecks**.

6. Desktop Virtualization

- Desktop environment is hosted on a central server.
- Users access desktops remotely over a network.
- Multiple virtual desktops run on one physical system.
- Provides centralized control and security.

Advantages:

- **Centralized management** of desktops.
- Access from **any device/location**.

Disadvantages:

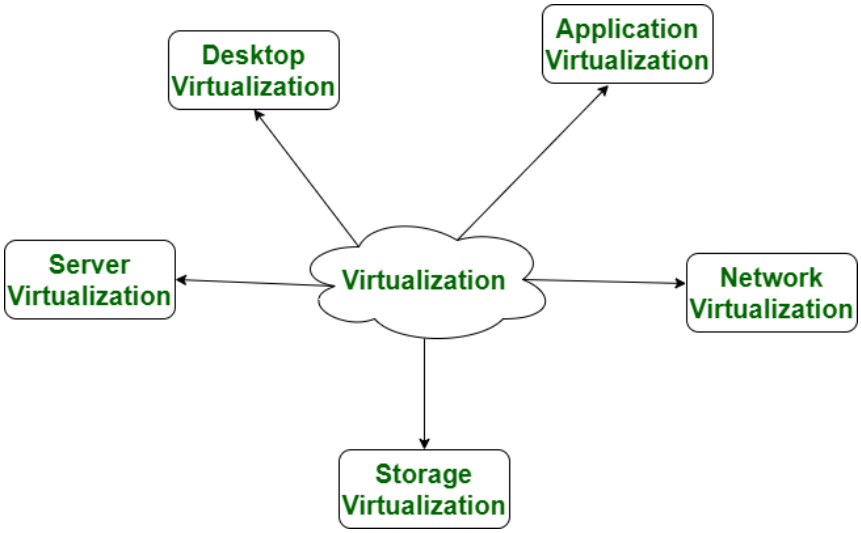
- Depends on **network connectivity**.
- May cause **latency issues**.

VIRTUALIZATION AND CLOUD COMPUTING

Cloud computing and virtualization are two fundamental concepts essential to modern IT infrastructure management. While they are often discussed together (and rely on each other), they serve diverse functions and provide unique operational benefits to a business.

What is Virtualization?

Virtualization is the core technology that serves as the establishment (foundation) of cloud computing. It empowers a single physical hardware framework to generate multiple, continuous digital environments.

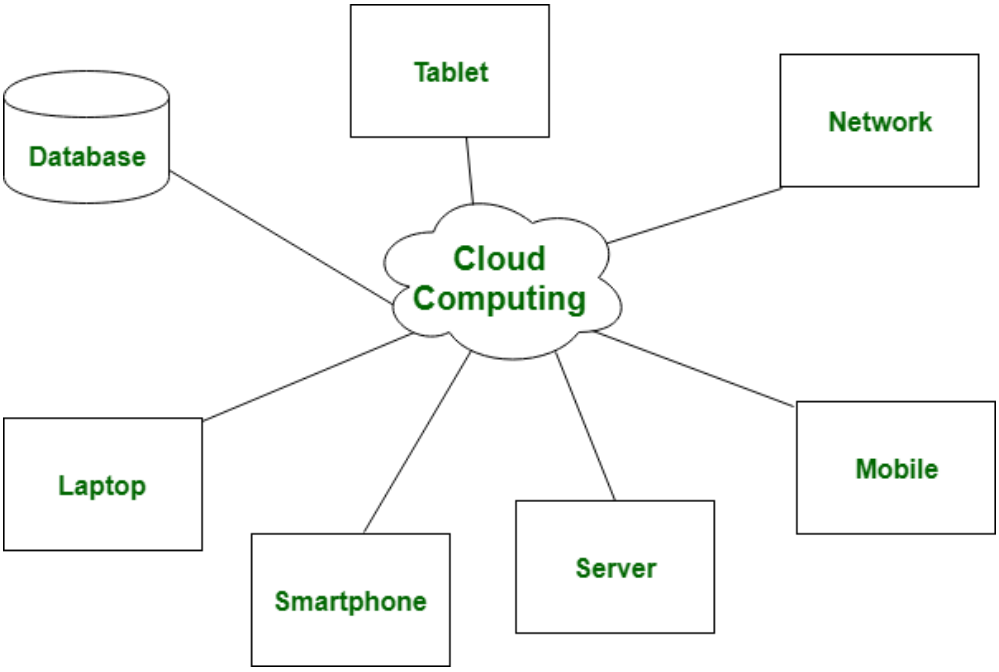


The Hypervisor: This is the essential software component that sits directly on the physical hardware, splitting it up to create several Virtual Machines (VMs).
Isolation: Every VM functions distinctly and independently. They do not interfere with one another, allowing different environments to run on the same physical server safely.
Types of Virtualization:

- **Hardware Virtualization:** The real, underlying physical hardware is separated into several distinct virtual machines.
- **Application Virtualization:** Programs are executed independently of the underlying hardware.

What is Cloud Computing?

Cloud computing is a highly accessible, **client-server computing architecture**. Instead of relying on a single physical machine you own, resources are centralized and delivered over the internet.



- **The Service Model:** It acts as a powerful business tool that provides pooled, automated resources that users can access entirely on-demand.
- **Pricing Model:** Cloud computing utilizes a **pay-as-you-go** model. Consumption is the metric for billing, meaning users only pay for exactly what they use.
- **Types of Cloud Computing:** The two primary categories are **Public Cloud** and **Private Cloud**.

Cloud Computing vs Virtualization

Aspect	Cloud Computing	Virtualization
Definition	Delivers computing resources over the internet as services	Creates multiple virtual machines from a single physical system
Basic Concept	Pools and distributes resources on demand	Divides one physical system into multiple virtual environments
Scalability	Very high (virtually unlimited)	Limited by hardware capacity
Flexibility	Highly flexible	Less flexible
Setup	Complex infrastructure required	Simple setup
Cost Structure	Low upfront cost, pay-as-you-go (OPEX)	High upfront cost (CAPEX), low maintenance cost
Infrastructure	Uses multiple distributed servers	Usually uses a single server
Workload Type	Stateless workloads	Stateful workloads
Configuration	Image-based configuration	Template-based configuration
Accessibility	Accessible via internet	Limited to internal network
Service Type	Provides IaaS, PaaS, SaaS	Not a service, only a technology
Integration	Easy expansion across regions and users	Limited expansion within same hardware

Disaster Recovery	Strong (data distributed across locations)	Weak (depends on single system)
Hardware Dependency	No need to own hardware	Requires physical hardware
Forms	Public, Private, Hybrid cloud	Hardware and Application virtualization

PROS AND CONS OF VIRTUALIZATION

Pros of Virtualization:

1. **Efficient Hardware Utilization:**
   Virtualization allows better use of hardware by running multiple virtual machines (VMs) on a single physical system. This reduces the need for additional physical hardware, lowering costs for both users and cloud service providers.

2. **High Availability:**
   Virtualization provides advanced features such as live migration and failover, ensuring that virtual machines remain available with minimal downtime.

3. **Easy and Efficient Disaster Recovery:**
   Data backup, recovery, and duplication become much simpler. Virtualization enables real-time backups and mirroring, increasing the chances of data recovery even after failures.

4. **Energy Saving:**
   By reducing the number of physical servers, virtualization decreases power consumption and cooling requirements, leading to lower operational costs.

5. **Quick and Easy Setup:**
   Unlike traditional systems that require hardware purchase and installation, virtual machines can be created and configured quickly, saving time and increasing productivity.

6. **Easy Cloud Migration:**
   Organizations can easily migrate their data and applications from physical servers to cloud environments, reducing maintenance and infrastructure costs.

7. **Resource Optimization:**
   Virtualization maximizes resource usage by efficiently allocating CPU, memory, and storage across multiple virtual machines.

Cons of Virtualization:

1. **High Initial Investment:**
   Setting up virtualization infrastructure (servers, storage, software) can be expensive initially, although it saves costs in the long run.
2. **Complexity in Management:**
   Managing multiple virtual machines and virtual networks can become complex, especially in large-scale environments.
3. **Security Risks:**
   Virtualization introduces additional layers, which can become targets for cyberattacks if not properly secured and monitored.
4. **Need for Skilled Professionals:**
   Organizations require trained staff to manage virtual and cloud environments, increasing training or hiring costs.
5. **Data Security Concerns:**
   Since data may be stored on shared or third-party infrastructure, there is a risk of unauthorized access or data breaches if proper security measures are not implemented.

TECHNOLOGY EXAMPLES(XEN,VMware)

Xen: paravirtualization:

- Xen is an **open-source virtualization platform** that has gained significant attention in the world of virtualization.
- It was initially developed at the **University of Cambridge** and is now maintained by the Xen Project, a collaborative community of developers.
- It is either used for **desktop or server virtualization**.
- Xen follows **paravirtualization** technology.
- **Paravirtualization** is a virtualization technique that involves modifying the operating system (OS) running inside a virtual machine (VM) to make it aware of its virtualized environment.
- Unlike full virtualization, where the guest OS is unaware of the virtualization layer and runs unmodified, paravirtualization requires changes to the guest OS to improve performance, security, and efficiency.
- Example: XenoLinux is a modified Linux OS that runs on Xen hypervisor.

Pros of Xen Hypervisor

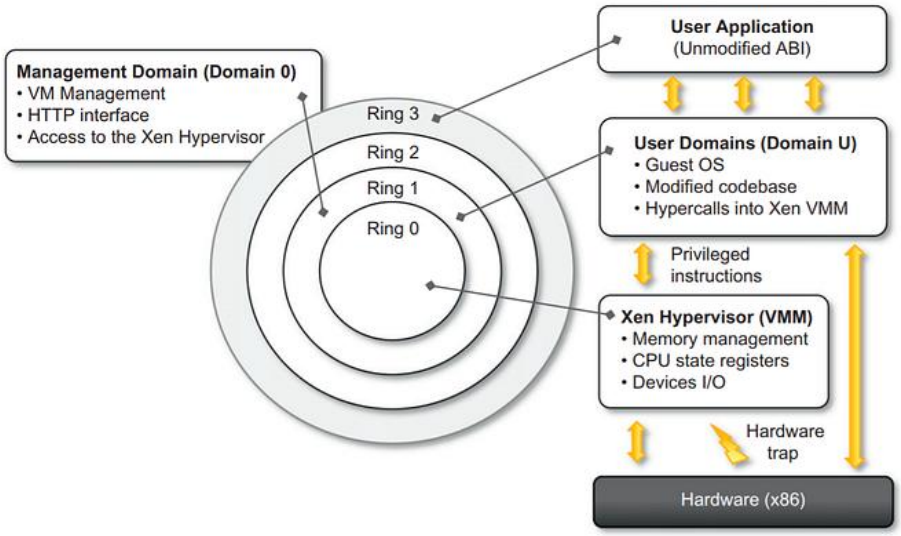
- **High Performance:**
  Uses paravirtualization to reduce overhead, giving better performance compared to full virtualization, especially for heavy workloads.
- **Strong Isolation:**
  Provides secure and stable environments by isolating each virtual machine, preventing failures or attacks from affecting others.

- **Open Source:**
  Freely available, reducing costs and allowing continuous improvements through community contributions.
- **Live Migration:**
  Enables moving VMs between physical machines with minimal downtime, useful for maintenance and load balancing.
- **Wide Compatibility:**
  Supports multiple operating systems as guests and hosts, making it flexible for different environments.

Cons of Xen Hypervisor

- **Complex Configuration:**
  Setup and management can be difficult for beginners, requiring more technical knowledge.
- **Limited Desktop Virtualization:**
  Not as optimized for desktop environments compared to solutions like VMware Horizon or Citrix XenDesktop.
- **Hardware Dependency:**
  Requires virtualization support (Intel VT-x / AMD-V); older systems may not perform well.
- **Limited Commercial Support:**
  Compared to VMware, it has fewer enterprise-level support resources.
- **Integration Challenges:**
  Can be harder to integrate with some tools or other virtualization platforms due to its paravirtualization approach.

The architecture of Xen is given below:



---

1. Hardware Layer (Bottom)

- This is the **physical machine (CPU, memory, devices)**.
- Xen directly interacts with hardware.

- Any privileged operation triggers a **hardware trap** to the hypervisor.

2. Xen Hypervisor (VMM) – Ring 0

- The core of the system.
- Runs at the **highest privilege level (Ring 0)**.
- Responsibilities:
  - Memory management
  - CPU scheduling
  - Device I/O control
- Acts as a **bridge between hardware and virtual machines**.

3. User Domains (DomU)

- These are **guest operating systems** (like Linux, Windows).
- Run in **less privileged rings (Ring 1/3)**.
- Key points:
  - OS is slightly **modified (paravirtualization)**
  - Uses **hypercalls** instead of direct hardware access
  - Cannot execute privileged instructions directly

4. Management Domain (Dom0)

- Special, most important VM.
- Has **direct access to the hypervisor**.
- Responsibilities:
  - VM creation and deletion
  - Resource allocation
  - Provides drivers and management tools
- Acts as a **controller of all other VMs (DomU)**.

5. User Applications

- Run on top of guest OS (DomU).
- **No modification required** (normal apps).
- Interact with OS like in a normal system.

6. Rings Concept (Security Levels)

- CPU privilege levels:
  - **Ring 0** → Hypervisor (most privileged)
  - **Ring 1/2** → Guest OS
  - **Ring 3** → User applications
- Ensures **security and isolation** between layers.

Flow of Execution

1. User application sends request
2. Guest OS (DomU) processes it
3. If hardware access needed → **hypercall to Xen**
4. Xen interacts with hardware
5. Result returned back to application

VMware: Full Virtualization

• VMware is a widely used virtualization platform in both industry and cloud environments.
• It is a commercial company known for providing powerful virtualization and cloud computing solutions.
• VMware is used for both desktop and server virtualization.
• VMware follows full virtualization technology.
• Full virtualization is a technique where the complete physical hardware is simulated, and each virtual machine runs as if it has its own real hardware.
• In this approach, the guest operating system does not need any modification and is completely unaware that it is running in a virtual environment.
• The hypervisor (like VMware ESXi) handles all interactions between the hardware and virtual machines.
• This approach provides better compatibility because any OS (Windows, Linux, etc.) can run without changes, but it may introduce slightly more overhead compared to paravirtualization.
• VMware also provides tools like VMware vSphere for managing virtual machines, monitoring performance, and handling resources efficiently.
• **Example:** Running Windows or Linux directly on VMware without modifying the OS.

Pros of VMware

• **Highly Reliable:**
VMware is known for its strong reliability and stability, making it suitable for enterprise environments where uptime is very important.

• **Mature Ecosystem:**
It has a large ecosystem with many third-party tools, integrations, and resources, making it easier to build and manage solutions.

• **High Performance:**
VMware's hypervisor, VMware ESXi, is optimized for speed and efficient resource utilization.

• **Strong Security:**
Provides advanced security features like VMware vSphere security to protect virtual machines and data.

• **Powerful Management Tools:**
Tools like **vCenter Server** allow centralized management, monitoring, and performance optimization of virtual environments.

• **Live Migration:**
Supports live migration (vMotion), allowing VMs to move between hosts with minimal downtime for maintenance and load balancing.

• **Wide OS Support:**
Supports many guest operating systems (Windows, Linux, etc.), making it flexible for different applications.

Cons of VMare

• **High Cost:**
VMware products are expensive, especially for small organizations. Licensing and subscription costs can increase quickly.

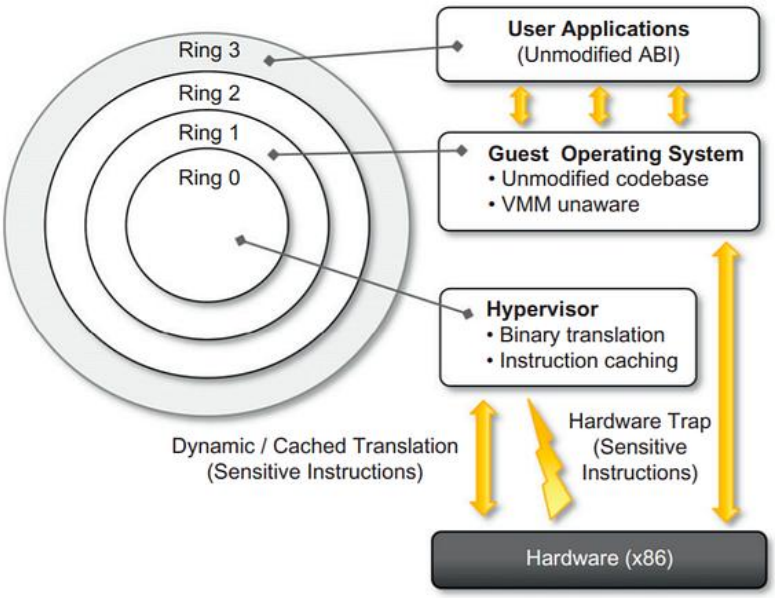
• **Resource Intensive:**
VMware requires powerful hardware and dedicated resources for the hypervisor and management tools, increasing infrastructure costs.

• **Complex Setup:**
Installation and management can be difficult for beginners and may require skilled IT professionals.

• **Vendor Lock-In:**
Once an organization adopts VMware solutions, switching to other platforms can be difficult and costly.

• **Limited Free Features:**
The free version of VMware vSphere Hypervisor has limited features, while advanced functionalities are only available in paid versions.

The architecture of VMware is given below:



1. Hardware Layer (Bottom)

- Represents physical hardware (CPU, memory, devices).
- The hypervisor directly interacts with this layer.
- Sensitive instructions generate a hardware trap to the hypervisor.

2. Hypervisor Layer (VMware ESXi)

- The core virtualization layer.

- Responsibilities:
    - CPU and memory management
    - Device handling
    - Resource allocation
- Uses:
    - **Binary Translation:** Converts sensitive instructions into safe ones
    - **Instruction Caching:** Stores translated instructions to improve performance

3. Guest Operating System

- Runs in a virtual machine.
- Unmodified codebase (no changes required).
- Completely unaware of virtualization (VMM unaware).
- Believes it is running on real hardware.

4. User Applications

- Run on top of the guest OS.
- Use normal system calls (no changes needed).
- Work exactly like in a physical system.

5. Rings Concept (Privilege Levels)

- **Ring 0:** Hypervisor (highest control)
- **Ring 1/2:** Guest OS (less privileged)
- **Ring 3:** User applications

This ensures security and isolation.

Working Flow

1. User application sends request
2. Guest OS processes it
3. If privileged instruction → trapped to hypervisor
4. Hypervisor uses binary translation
5. Hardware executes the instruction
6. Result returned back

BUILDING BLOCKS OF CONTAINERS

Introduction:

Containers are lightweight virtualization units that package an application along with all its dependencies, ensuring portability and consistency across different environments. They are widely used in cloud computing for scalable and efficient deployment of applications.

The concept of container building blocks refers to the essential components required to create, manage, and scale containers. These building blocks are broadly classified into two categories:

1. **Lifecycle Management** – deals with a single container
2. **Cluster Management** – deals with multiple containers across hosts

The container engine acts as a connecting element between these two.

"These building blocks form a taxonomy of the container ecosystem."

I. Lifecycle Management Building Blocks:

Lifecycle management components are responsible for managing the creation, execution, and termination of individual containers.

1. Runtime Engine

The runtime engine provides the interface for managing the lifecycle of containers. It is responsible for creating, starting, stopping, and deleting containers. It also handles resource allocation, device mapping, and interaction between the container and the host system.

The runtime engine uses container images as input and relies on drivers for low-level execution. Examples include Docker, rkt, and LXD.

2. Driver

A driver is responsible for interacting with the operating system kernel and executing low-level container functionalities. It implements core mechanisms such as namespaces and control groups (cgroups), which enable isolation and resource management.

Examples of drivers include runC, LXC, and OpenVZ. The driver acts as a bridge between the runtime engine and the underlying OS.

3. Repository

A repository is a centralized system used to store, manage, and distribute container images and their descriptions. It provides a version-control-like interface that allows users to publish and retrieve images.

DockerHub is a common example of a container repository. It enables easy sharing and reuse of container images across different environments.

4. Container Image and Image Description

A container image is a packaged file system that contains application code, libraries, and dependencies required to run an application. These images are read-only and are built using layered architecture.

An image description, such as a Dockerfile, defines the steps required to create an image. The runtime engine uses this description to build and execute containers. Images can also be created from existing containers using snapshots.

II. Cluster Management Building Blocks:

Cluster management components are used to manage containers across multiple machines in a distributed environment.

1. Orchestrator

An orchestrator manages the deployment, scaling, and coordination of containers across multiple hosts. It groups containers into logical units (such as pods) and ensures they run efficiently.

Kubernetes is a widely used orchestrator that automates container management in large-scale systems.

2. Discovery Service

A discovery service enables containers to find and communicate with each other in a dynamic environment. Since containers are frequently created and destroyed, the discovery service maps static service names to dynamically changing container instances.

This ensures seamless communication between distributed components.

3. Overlay Network

An overlay network provides virtual networking that allows containers to communicate across different hosts. It abstracts the underlying physical network and ensures smooth

communication between containers, regardless of their location. Technologies like Flannel and Calico are commonly used for overlay networking.

4. Scheduler

The scheduler is responsible for assigning containers to appropriate hosts based on resource availability and constraints. It ensures efficient utilization of resources, load balancing, and proper distribution of workloads across the cluster.

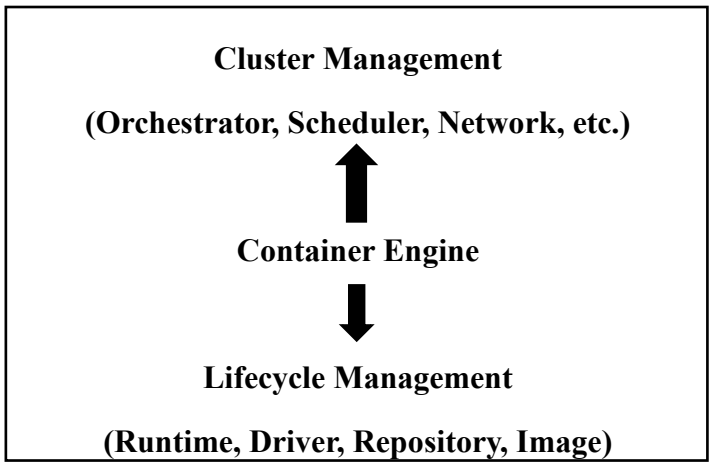
5. State Storage

State storage maintains the configuration and current state of the container cluster. It stores information required for scheduling, discovery, and orchestration.

It is typically implemented using distributed key-value stores such as etcd, Consul, or Zookeeper to ensure consistency and scalability.

6. Resource Monitor

A resource monitor collects and analyzes data related to resource usage such as CPU, memory, and system health. It helps administrators monitor performance and detect issues in the cluster. Tools like cAdvisor and Docker stats are commonly used for this purpose.



III. Supporting Internal Mechanisms

In addition to the main building blocks, containers rely on several internal operating system features for their efficient functioning. These mechanisms provide isolation, resource control, storage efficiency, and communication support.

1. Namespaces (Isolation Mechanism)

Namespaces provide isolation between containers by creating separate environments for each container. Each container gets its own view of system resources such as processes, network interfaces, and file systems.

For example, a process running inside one container cannot see or interact with processes in another container. This ensures security and independence of containers.

2. Control Groups (cgroups) (Resource Management)

Control groups, or cgroups, are used to manage and limit the resource usage of containers. They control how much CPU, memory, disk I/O, and other resources a container can use.

This prevents any single container from consuming excessive resources and ensures fair allocation among multiple containers.

3. Union File System (Layered Storage)

The union file system allows containers to use a layered file structure. A container image consists of multiple layers, such as base OS, libraries, and application code.

When a container runs, a writable layer is added on top of these read-only layers. This improves storage efficiency and allows reuse of common layers across multiple containers.

4. Volumes (Persistent Storage)

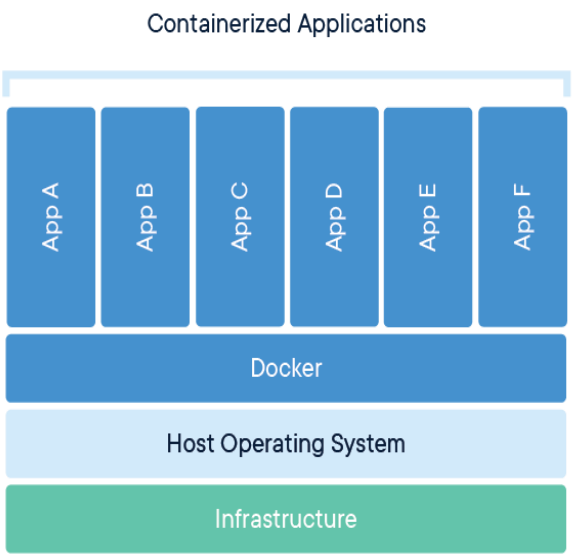
Containers are temporary by nature, so volumes are used to store data permanently. Volumes allow data to persist even if a container is stopped or deleted.

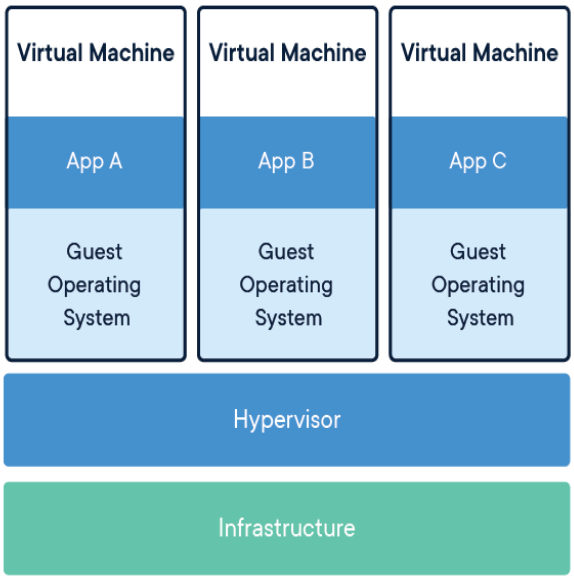
They are commonly used for databases, logs, and user data.

5. Networking (Communication)

Container networking enables communication between containers, between containers and the host, and across multiple hosts.

Different networking modes such as bridge, host, and overlay networks are used depending on the requirements.





Container Platforms (LXC, Docker)

Definition

Container platforms are software environments that allow applications to run in isolated units called containers, which share the host operating system but behave like separate systems. Examples include LXC (Linux Containers) and Docker.

1. LXC (Linux Containers)

Definition:

LXC is an OS-level virtualization technology that creates multiple isolated Linux systems on a single host.

Important Parts of LXC

1. Namespaces

Definition:

Namespaces isolate system resources for each container.

Explanation:
Each container gets its own view of system resources.
Types:

1. PID namespace – separates processes
2. Network namespace – separate IP, ports
3. Mount namespace – separate file system
4. UTS namespace – separate hostname
5. IPC namespace – separate communication

Important Point: Without namespaces, containers cannot be isolated.

2. Control Groups (cgroups)

Definition:
cgroups control and limit resource usage of containers.
Explanation:
They ensure one container does not use all CPU or memory.
Controls:

1. CPU usage
2. Memory usage
3. Disk I/O
4. Network bandwidth

Important Point: cgroups provide resource management.

3. Filesystem Isolation

Definition:
Each container has its own file system.
Explanation:
Containers do not affect host files or other containers.
Example: One container cannot access another container's files.

4. Networking

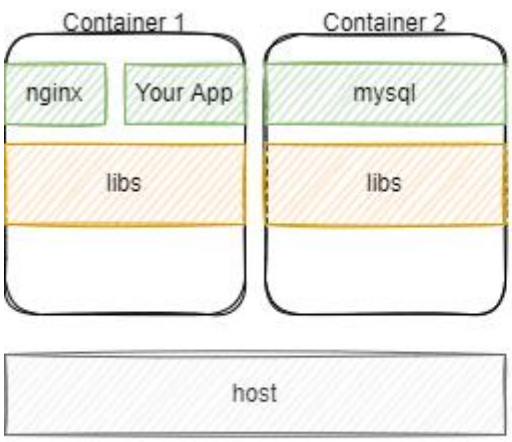
Definition:
Provides separate network environment for each container.
Explanation:
Each container can have its own IP address and ports.

Advantages of LXC

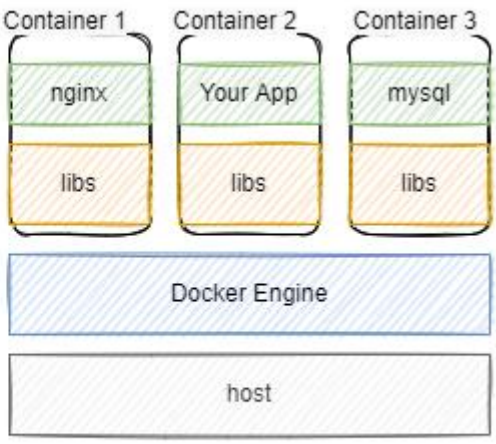
1. Lightweight (no separate OS needed)
2. Fast startup
3. Efficient resource usage
4. Suitable for running multiple OS environments

Example

Running Ubuntu, CentOS containers on one Linux machine using LXC



LXC



Docker

2. Docker

Definition

Docker is a platform that packages applications and dependencies into containers for easy deployment.

Important Parts of Docker (Very Important)

1. Docker Engine

Definition:

The core software that runs and manages containers.

Explanation:

Acts as the brain of Docker.

2. Docker Image

Definition:

A read-only template used to create containers.

Explanation:

Contains application code, libraries, dependencies.

Example: Image of a Java app

3. Docker Container

Definition:

A running instance of a Docker image.

Explanation:

It is the actual working application.

Important: Image = blueprint, Container = execution

4. Docker Hub

Definition:

Online repository to store and share Docker images.

Explanation:

You can download ready-made images.

Working Procedure of Docker

1. Write Dockerfile (instructions)
2. Build Docker image

3. Store image in Docker Hub
4. Run container from image

Advantages of Docker

1. Easy to use
2. Portable across systems
3. Fast deployment
4. Supports microservices

Example

Running a Node.js web app inside a Docker container

LXC vs Docker

Clear Understanding:

1. LXC runs full OS containers
2. Docker runs application containers
3. Docker is simpler and developer-friendly
4. LXC is closer to virtual machines

Important Highlights :

1. Namespaces = Isolation
2. cgroups = Resource control
3. Docker Image ≠ Container
4. Docker is widely used in industry
5. LXC is base technology for containers

Difference between LXC and Docker :

Parameter	LXC	Docker
Developed by	LXC was created by IBM, Virtuozzo, Google and Eric Biederman.	Docker was created by Solomon Hykes in 2003.
Data Retrieval	LXC does not support data retrieval after it is processed.	Data retrieval is supported in Docker.
Usability	It is a multi-purpose solution for virtualization.	It is single purpose solution.
Platform	LXC is supported only on Linux platform.	Docker is platform dependent.

Parameter	LXC	Docker
Virtualization	LXC provides us full system virtualization.	Docker provides application virtualization.
Cloud support	There is no need for cloud storage as Linux provides each feature.	The need of cloud storage is required for a sizeable ecosystem.
Popularity	Due to some constraints LXC is not much popular among the developers.	Docker is popular due to containers and it took containers to a next level.
Speed Of Deployment	LXC is not lightweight and consumes a lot of time .	Docker Containers are lightweight and fast

Container Orchestration

Definition:

Container orchestration is the process of automating the deployment, management, networking, and scaling of containers so applications can run efficiently at large scale

Need for Container Orchestration :

Explanation:

As applications grow, they use many containers (sometimes hundreds or thousands), which become difficult to manage manually.

Problems Solved

1. Managing containers across multiple servers
2. Handling failures automatically
3. Scaling applications based on demand
4. Running multiple versions (testing, production)
5. Maintaining application availability

Important Point: Orchestration removes manual effort and complexity.

Uses / Use Cases :

1. Managing containers across multiple machines
2. Running large applications with many microservices
3. Running duplicate containers (replicas) for reliability
4. Deploying applications across different regions
5. Optimizing resource usage and cost

Benefits of Container Orchestration :

1. Automation

Definition:
Automatically manages container lifecycle (creation, deployment, deletion)

2. Load Balancing

Definition:
Distributes traffic across containers for better performance

3. High Availability

Definition:
Ensures application runs continuously even if one container fails

4. Resource Optimization

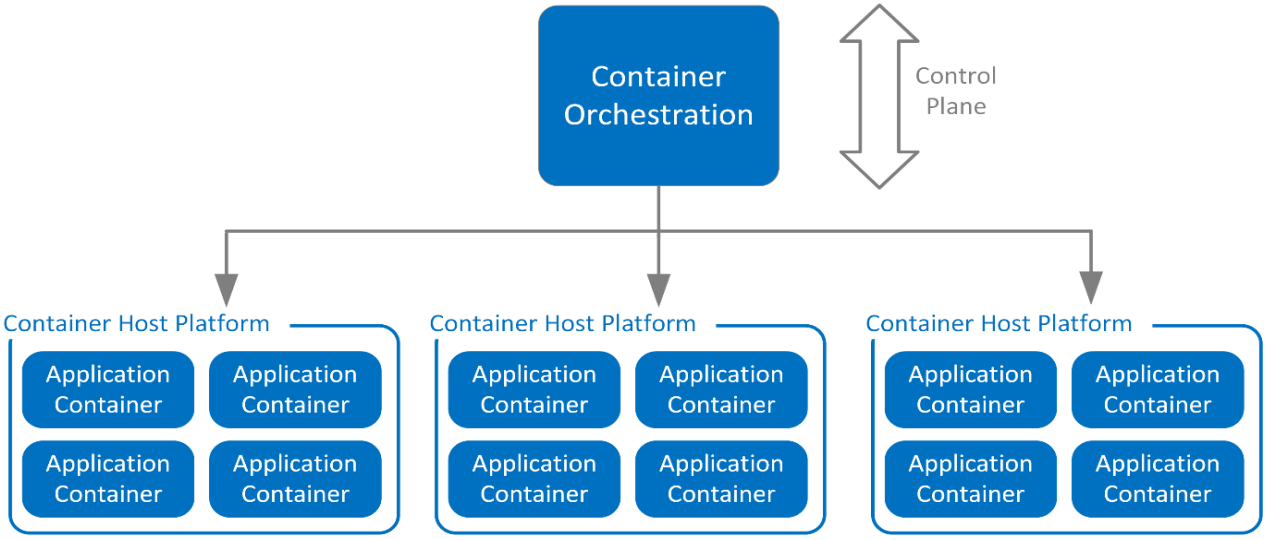
Definition:
Efficiently uses CPU, memory, and storage

5. Performance Improvement

Definition:
Automatically adjusts containers for best performance

6. Monitoring

Definition:
Tracks container health and system status.



Working of Container Orchestration :

Explanation

Container orchestration works by managing containers across multiple servers (nodes) in a cluster.

Procedure

1. Containers are created using tools like Docker
2. Configuration file (YAML/JSON) defines requirements
3. Orchestration tool reads configuration
4. Containers are scheduled on suitable nodes
5. System manages networking between containers
6. Monitors performance and health
7. Automatically scales or restarts containers

Important Point: Cluster = group of nodes managing containers

Key Concepts

1. Cluster

Definition:

Group of servers working together to run containers

2. Node

Definition:

A single machine (physical or virtual) in the cluster

3. Control Plane

Definition:

Main controller that manages the orchestration system

4. Configuration File

Definition:

File that defines how containers should run

Advantages:

1. Reduces manual work
2. Improves scalability
3. Ensures reliability
4. Saves cost through efficient resource usage
5. Supports large-scale applications

Challenges :

1. Requires skilled professionals
2. Complex to manage at large scale
3. Needs proper configuration and monitoring
4. Additional system overhead

Example

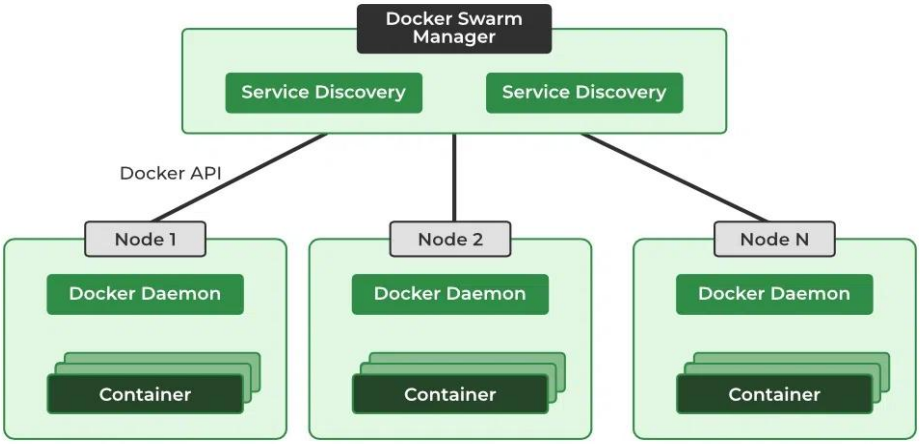
Running a large e-commerce application with hundreds of containers across multiple servers, automatically scaling during high traffic

DOCKER SWARM AND KUBERNETES

Definition

In cloud computing, Docker Swarm and Kubernetes use a structured architecture to manage containers across multiple systems.

Docker Swarm Architecture:



Architecture Components

1**. Docker Swarm Manager :**

Definition:

Central controller of the swarm cluster

Functions:

1. Handles Docker API requests
2. Performs service discovery
3. Schedules tasks to nodes

Important: Manager is the brain of the system

2. Service Discovery

Definition:

Mechanism to identify and connect services across nodes

Explanation:

1. Helps containers communicate
2. Ensures proper routing of requests

3. Nodes (Node 1, Node 2, Node N)

Definition:

Machines participating in swarm

4. Docker Daemon

Definition:

Software running on each node to manage containers

Explanation:

1. Receives instructions from manager
2. Runs containers

5. Containers

Definition:

Actual running applications inside nodes

Architecture Working

1. User sends request through Docker API
2. Manager receives and processes request
3. Manager performs service discovery

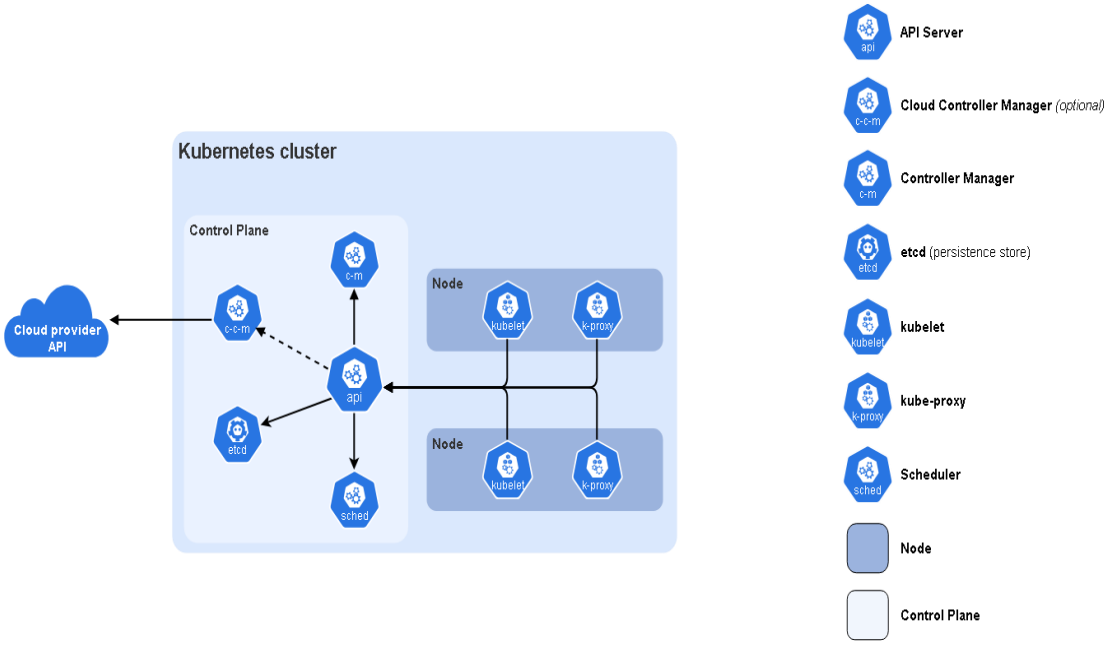
4. Tasks are assigned to nodes
5. Docker daemon runs containers
6. Output is returned to user

KUBERNETES

Definition

Kubernetes is an open-source container orchestration platform used to automate deployment, scaling, and management of containerized applications in cloud environments

Kubernetes Architecture



Architecture Components

1. Control Plane (Master Node)

Definition:

Central controller that manages the entire Kubernetes cluster

Main Components of Control Plane

a. API Server

Definition:

Entry point for all user requests

b. Scheduler

Definition:

Assigns Pods to suitable nodes

c. Controller Manager

Definition:

Maintains desired system state

d. etcd

Definition:

Stores cluster configuration and data

2. Worker Node

Definition:
Machine where applications run

Components of Worker Node

1. **Kubelet** – manages containers on node
2. **Container Runtime** – runs containers
3. **Kube Proxy** – handles networking

3. Pod

Definition:
Smallest unit in Kubernetes that contains one or more containers

Working (Procedure):

1. User sends request to API Server
2. API Server processes request
3. Scheduler assigns Pods to nodes
4. Kubelet runs containers inside Pods
5. Controller monitors and maintains system
6. System automatically scales and heals

Advantages

1. Automatic scaling
2. Self-healing
3. High availability
4. Efficient resource usage
5. Suitable for large-scale systems

Example

Running a large cloud application with auto-scaling and failure recovery using Kubernetes

Docker Swarm vs Kubernetes

The following are the docker swarm and Kubernetes:

Kubernetes	Docker Swarm
Kubernetes is an open-source platform used for maintaining and deploying a group of containers	Docker Swarm is designed to be a simple and lightweight container orchestration.
In practice, Kubernetes is most commonly used alongside Docker for better control and implantation of containerized applications.	With Docker swarm, multiple containers run on the same hardware much more efficiently than the VM environment & productivity of Docker is extremely high.
Applications are deployed as a combination of pods, Deployment, and services.	Apps are deployed in the form of services.

Kubernetes	Docker Swarm
It supports auto-scaling of the container in a cluster. With more efficient	Docker Swarm does support auto-scaling but is not as efficient as docker swarm.
The health check is of two kinds: liveness and readiness.	Health checks are limited to service.
Hard to set up and configure.	Docker swarm setup and installation are easy.
It does not have extensive documentation but is quite less than Docker Swarm. But it does include everything from installation to deployment.	Docker swarm documentation is more effective, more extensive, and has even more capabilities & it includes everything from installation to deployment & quick-start instructions as well as a more detailed tutorial.
Kubernetes installation is provided to be quite difficult than Docker swarm and even the command for Kubernetes is quite more complex than Docker swarm.	Docker swarm installation is quite easier, by using fewer commands you can install Docker in your virtual machine or even on the cloud.
Azure, buffer, intel, Evernote, and Shopify Using Kubernetes.	Citizens Bank, and MetLife companies using Docker swarm.

PUBLIC CLOUD VM

Definition

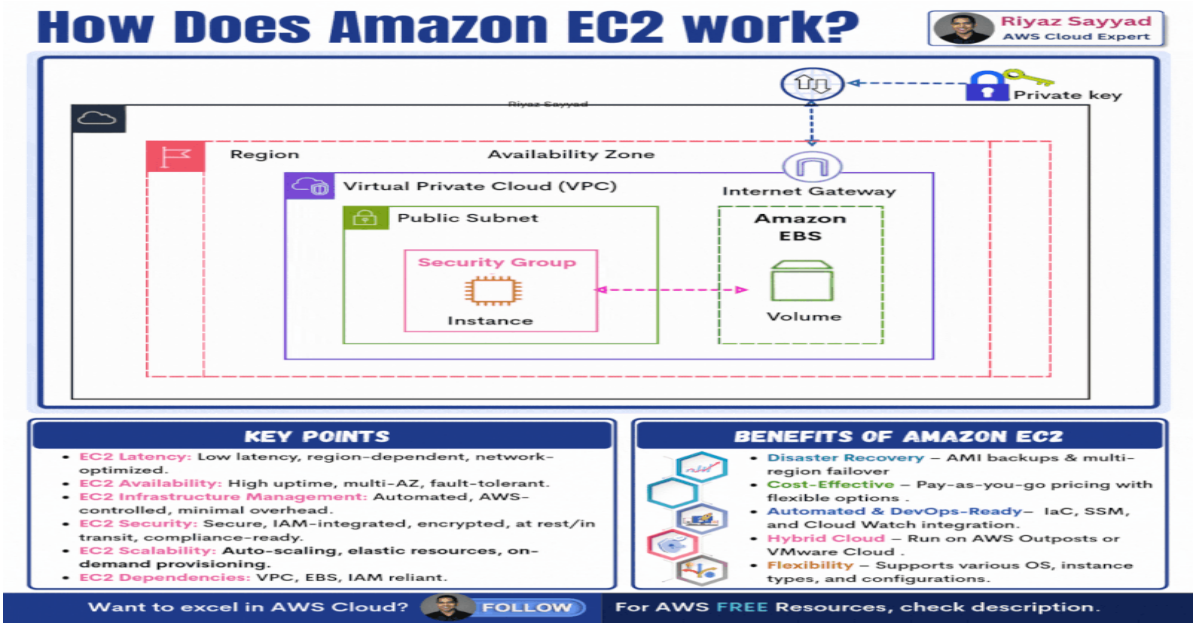
A Public Cloud Virtual Machine (VM) is a virtual server provided over the internet by a cloud provider, allowing users to run applications without owning physical hardware.

Amazon EC2

Definition

Amazon EC2 (Elastic Compute Cloud) is a public cloud service that provides scalable virtual servers to run applications.

Architecture of EC2



Main Components

1. EC2 Instance

Definition:

A virtual server used to run applications

Explanation:

1. Acts like a real computer
2. Runs OS (Linux/Windows)
3. Used to host apps, websites

2. AMI (Amazon Machine Image)

Definition:

Template used to launch EC2 instances

Explanation:

1. Contains OS, software, configuration
2. Helps quick deployment
3. Can create custom AMI

3. Instance Type

Definition:

Specifies hardware configuration

Explanation:

1. CPU, RAM, storage
2. Types like General, Compute, Memory optimized

4. VPC (Virtual Private Cloud)

Definition:

Virtual network for EC2 instances

Explanation:

1. Provides isolation
2. Controls IP range

3. Acts like private network

5. Subnet

Definition:

Subdivision of VPC

Explanation:

1. Public subnet → internet access
2. Private subnet → no direct internet

6. Security Groups

Definition:

Firewall that controls inbound and outbound traffic

Explanation:

1. Allows or blocks ports (e.g., 22 for SSH, 80 for HTTP)
2. Instance-level security

7. Storage (EBS – Elastic Block Storage)

Definition:

Persistent storage attached to EC2

Explanation:

1. Data remains even after instance stops
2. Used like hard disk

8. Internet Gateway

Definition:

Connects VPC to internet

Explanation:

1. Enables public access
2. Required for web applications

Additional Components:

9. Elastic IP

Definition:

Static public IP address

10. Key Pair

Definition:

Security credential used to access instance

11. Load Balancer

Definition:

Distributes traffic across multiple instances

12. Auto Scaling

Definition:

Automatically increases/decreases instances

Working (Procedure):

1. User selects AMI
2. Chooses instance type
3. Configures VPC and subnet
4. Attaches security groups
5. Adds storage (EBS)

6. Launches EC2 instance
7. Connects using SSH/Remote Desktop
8. Deploys application
9. Traffic flows via Internet Gateway

Advantages

1. Scalable resources
2. Pay-as-you-go pricing
3. High availability
4. Flexible configuration
5. Easy deployment
6. Global infrastructure

Example

Launching a Linux EC2 instance, installing Apache server, and hosting a website accessible via internet

Difference between Private, Public and Hybrid cloud

Private Cloud	Public Cloud	Hybrid Cloud
Private cloud infrastructure is hosted by organization and it is isolated for outside access.	Public cloud infrastructure is hosted by third party service provider which can be access by any public user.	Hybrid cloud consists of infrastructure which is hosted in public and private cloud model.
Infrastructure is completely managed by organization recurring cost.	Infrastructure is managed by third party provider making users only pay for the resource usage.	Infrastructure is maintained by both organization and provider making flexible cost.
More security and control over data as infrastructure is isolated to organization.	Security is managed by third party provider and less control over data as data is stored as providers infrastructure.	Flexible for storing data and managing security.
More usage cost as resources has to be managed on own.	Less usage cost as users have to only pay for what they use.	Cost occurs for private infrastructure while public infrastructure is less costly.

Private Cloud	Public Cloud	Hybrid Cloud
Limited in terms of scalability as limited infrastructure.	Highly scalable as more resources are available.	Scalable due to implementation of both models.
Availability depends on infrastructure locality	More availability as infrastructure is mostly spread across various regions.	Allows high availability because of private and public infrastructure.

CONTAINER OFFERINGS:

What is a Container?

A container is a lightweight virtual environment that packages an application and its dependencies to run consistently across different computing environments.
App + Libraries + OS dependencies = **Container**
Containers provide an efficient and portable way to run applications by packaging them with all required dependencies, ensuring consistency across different environments.

Container Offerings:

Container offerings are cloud-based services that provide infrastructure and tools to deploy, manage, and scale containerized applications.
Examples: AWS ECS, Google GKE, Azure Container Instances

Why we need Container offerings?

When we use containers, we still need to:

- Set up servers
- Install software (like Docker)
- Run and monitor containers
- Handle failures and scaling

Doing all this manually is complex, time-consuming, and error-prone.

How Container Offerings Help:

- **Easy Deployment** : Quickly run containers without manual setup.
- **Automatic Scaling** : Increases or decreases containers based on demand.
- **Reduced Management** : Cloud provider handles infrastructure.
- **High Availability** : Automatically restarts failed containers.
- **Cost Efficiency** : Uses resources efficiently, reducing cost.

Container offerings reduce complexity by automating deployment, scaling, and management of containers, making applications more reliable and efficient.

AMAZON ELASTIC CONTAINER SERVICE(ECS):

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service that helps to run, manage, and scale containerized applications on AWS.

Amazon ECS is a service by AWS that:

- Runs your containers
- Manages them
- Scales them

Working of Amazon ECS:

The working of ECS involves the following steps:

1. **Application Development:**
   First, the developer creates an application such as a web app, mobile backend, or API. This application includes code, libraries, and configuration files.
2. **Containerization using Docker:**
   The application is then packaged into a container using Docker. The container includes:
   - Application code
   - Required libraries
   - Dependencies

   This ensures the application runs consistently across different environments.

3. **Uploading to AWS:**
   The container image is uploaded to a container registry such as:
   - Amazon Elastic Container Registry (ECR)

   The image acts as a blueprint for running the application.
4. **Running Containers using ECS:**
   ECS uses a task definition to run the container.
   A task definition specifies:
   - Container image
   - CPU and memory requirements
   - Network settings

   ECS then launches the container on cloud infrastructure (EC2 or Fargate).
5. **Monitoring and Management:**
   ECS continuously monitors the container to ensure it is running properly. If any container fails or crashes:
   - ECS automatically restarts it
   - Ensures high availability
6. **Automatic Scaling:**
   ECS supports auto-scaling based on user demand.
   - If traffic increases → more containers are launched
   - If traffic decreases → extra containers are stopped

   This improves performance and reduces cost.

Example:

Consider an online shopping website:

- The application is containerized using Docker
- Uploaded to AWS
- ECS runs the containers
- During high traffic, ECS automatically increases containers
- If any container fails, ECS restarts it

Core Functions of Amazon ECS:

Amazon ECS mainly performs four important functions:

1. Running containers
2. Scaling, Monitoring
3. Managing infrastructure

1. Running Containers:

ECS is responsible for running containerized applications on cloud infrastructure. It uses task definitions to launch containers with specified CPU, memory, and network settings.

Containers can be run using EC2 instances or Fargate (serverless mode).

2. Automatic Scaling:

ECS automatically scales the number of containers based on demand.

- If user traffic increases, ECS launches additional containers.
- If traffic decreases, ECS stops extra containers. This ensures efficient resource utilization and better performance.

3. Monitoring and Health Checking:

ECS continuously monitors the health and status of containers. It checks whether containers are running properly and responding to requests. If any container fails or becomes unhealthy, ECS automatically restarts it.

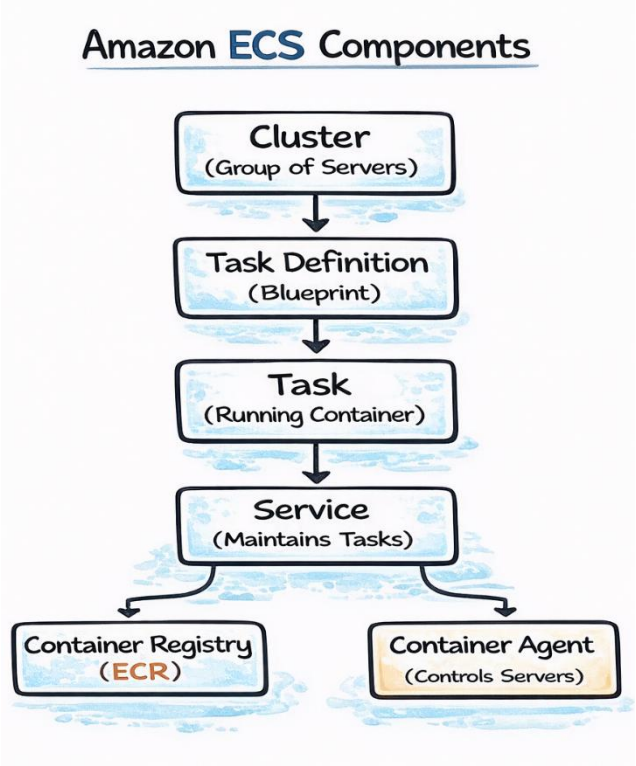
4. **Load Balancing:**
   ECS distributes incoming user requests across multiple containers using load balancers. This prevents overloading of a single container and ensures faster response time.
5. **Infrastructure Management:**
   ECS manages the underlying infrastructure required to run containers.
   - In EC2 mode, users manage servers.
   - In Fargate mode, AWS manages servers completely.
     This reduces manual effort and simplifies deployment.
6. **Service Management:**
   ECS ensures that a specified number of containers are always running. If any container stops, ECS replaces it automatically to maintain availability.
7. **Security Management:**
   ECS provides security features such as:
   - Access control using IAM roles
   - Network isolation
   - Secure communication

Components of Amazon ECS:

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service provided by AWS that enables users to run and manage containerized applications efficiently. It consists of several components and supports different modes for executing containers.

1. **Cluster:**
   A cluster is a logical grouping of resources such as EC2 instances or Fargate where containers are deployed and executed. It acts as the environment in which applications run.
2. **Task Definition:**
   A task definition is a blueprint that describes how a container should run. It includes details such as:
   - Container image
   - CPU and memory requirements
   - Network and port configurations
3. **Task:**
   A task is a running instance of a task definition. It represents an active container executing the application.
4. **Service:**
   A service ensures that a specified number of tasks are always running. If any task fails or stops, ECS automatically replaces it to maintain availability.
5. **Container Agent:**
   The container agent is a software component that runs on EC2 instances and communicates with ECS. It helps ECS manage and control containers on the instances.

6. **Container Registry (ECR):**
   Amazon Elastic Container Registry (ECR) is used to store and manage container images. ECS retrieves container images from ECR to run applications.



Modes of Amazon ECS:

Amazon Elastic Container Service (ECS) provides different modes (also called launch types) to run containerized applications. These modes determine how the underlying infrastructure is managed. The two main modes of ECS are EC2 launch type and Fargate launch type.

1. EC2 Launch Type
2. Fargate Launch Type

1. EC2 Launch Type:

In EC2 launch type, containers run on Amazon EC2 virtual machines, and the user is responsible for managing the underlying servers.

Features:

- Users must create and manage EC2 instances
- Full control over infrastructure
- Requires installation and maintenance of Docker and ECS agent
- Flexible configuration of networking and storage

Advantages:

- Greater control over resources
- Suitable for customized environments
- Can optimize cost with reserved instances

Disadvantages:

- Requires server management

- More complex setup
- Time-consuming maintenance

Example:

A company running a large application that requires specific hardware or custom configurations may prefer EC2 mode to have full control over the environment.

2. Fargate Launch Type:

Fargate is a serverless execution mode where AWS manages the infrastructure, and users only need to define container requirements.

Features:

- No need to manage servers
- Automatically provisions resources
- Simplified deployment
- Integrated scaling and monitoring

Advantages:

- Easy to use
- No infrastructure management
- Faster deployment
- Reduced operational overhead

Disadvantages:

- Less control over infrastructure
- Slightly higher cost for some workloads
- Limited customization

Example:

A startup deploying a web application can use Fargate to run containers without worrying about server setup and maintenance.

Importance of Amazon ECS:

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service that simplifies the deployment and management of containerized applications. It plays a crucial role in modern cloud computing by improving efficiency, scalability, and reliability.

- **Easy Deployment**
  Allows quick deployment of containerized applications without complex setup.
- **Saves Time**
  Automates tasks like deployment, scaling, and monitoring, reducing manual effort.
- **Automatic Scaling**
  Increases or decreases containers based on user demand.
- **High Reliability**
  Automatically restarts failed containers, ensuring continuous availability.
- **Reduced Workload**
  Eliminates the need to manage infrastructure, especially in Fargate mode.

- **Efficient Resource Utilization**
  Uses CPU and memory efficiently, reducing wastage.
- **Cost Effective**
  Helps reduce costs by scaling resources only when needed.
- **Integration with AWS Services**
  Easily integrates with services like ECR, CloudWatch, and Load Balancer.
- **Improved Performance**
  Load balancing distributes traffic, ensuring faster response times.
- **Supports Modern Applications**
  Ideal for microservices, DevOps, and cloud-native applications.